

소프트웨어공학

Homework #3-1

2019060546

주원웅

두 설계의 목적

두 설계의 공통된 목적은 고객 리스트에 대한 관리와 이벤트 처리이다.

1. 고객 추가

- 고객 리스트에 고객을 추가하면서, 관련된 마케팅 작업 및 GUI 업데이트를 수행.

2. 이벤트 처리

- 고객 추가, 삭제, 변경 등과 같은 이벤트를 다른 클래스(MarketingLetters, CustomerManagementDialog)로 전달.

하지만 구현 방법에서 차이가 있으며, 이는 유연성, 확장성, 유지보수성에서 큰 차이가 있다.

첫 번째 설계 직접 참조 방식

CustomerList 클래스는 직접적으로 **MarketingLetters**와 **CustomerManagementDialog** 객체를 참조하며, 각각의 메서드를 호출하여 이벤트를 처리한다.

- 고객 추가 시, **MarketingLetters** 의 **customerAdded** 메서드와 **CustomerManagementDialog**의 **gui.list.add** 메서드를 호출.

장점

1. 간단한 구현

- 설계가 단순하고 초기 구현이 쉽다. 모든 호출이 CustomerList 내부에서 이루어 지므로 코드의 흐름이 명확하다.

2. 빠른 실행

- 직접 참조를 통해 이벤트를 처리하므로 호출 과정에서의 성능 손실이 적다.

문제점

1. highCoupling

- CustomerList는 MarketingLetters 및 CustomerManagementDialog와 강하게 결합되어 있다.
- 이로 인해, MarketingLetters나 CustomerManagementDialog 클래스가 변경되면 CustomerList도 수정이 필요하다.

2. 확장성 부족

- 새로운 이벤트 처리를 추가하려면 CustomerList의 메서드에 직접 호출 코드를 추가해야 한다.

3. 테스트 어려움

- CustomerList를 테스트하려면 MarketingLetters와 CustomerManagementDialog 클래스도 함께 테스트해야 한다.
- 단일 클래스의 독립적인 테스트가 어렵다.

4. 재사용성 부족

- CustomerList가 특정 클래스에 종속적이므로, 다른 시스템에서 재사용하기 어렵다.

두 번째 설계 이벤트 핸들러 기반

CustomerList는 CustomerEventHandler라는 인터페이스를 사용하여 이벤트를 처리.

CustomerEventHandler를 구현하는 클래스(MarketingLetters, CustomerManagementDialog)가 실제 이벤트 동작을 정의.

CustomerList는 이벤트가 발생할 때 등록된 핸들러 리스트를 순회하며 각 핸들러의 메시지를 호출.

장점

1. Low Coupling

- CustomerList는 CustomerEventHandler 인터페이스만 알고 있으며, 구체적인 구현체(MarketingLetters, CustomerManagementDialog)에 대해 알 필요가 없다.

2. 확장성

- 새로운 이벤트 핸들러(예: CustomerLogger)를 추가하려면 단순히 CustomerEventHandler를 구현하는 클래스를 추가하고 핸들러 리스트에 등록하면 된다.
- CustomerList를 수정할 필요가 없다.

3. 테스트 용이성

- CustomerList를 독립적으로 테스트할 수 있다. 필요한 경우, 실제 핸들러 대신 Mock 객체를 주입하여 테스트 가능.

4. 재사용성

- CustomerList가 특정 클래스에 종속적이지 않으므로, 다른 시스템에서 쉽게 재사용할 수 있다.

5. 이벤트 기반 설계

- 이벤트 기반 구조를 통해 코드의 의도를 더 명확히 표현할 수 있다.
- "고객이 추가되었을 때 발생하는 동작"을 명확히 분리.

문제점

1. 초기 설계의 복잡성 증가

- 인터페이스 정의, 핸들러 등록 및 관리 등 초기 설계가 더 복잡해질 수 있다.

2. 성능 저하 가능성

- 이벤트 핸들러를 순회하면서 호출하므로, 이벤트가 매우 많아질 경우 성능에 영향을 줄 수 있다. 그러나 이는 일반적인 수준에서는 무시할 수 있는 문제다.

비교

두번째 설계가 더 좋다고 생각한다.

Low Coupling, 확장성, 유지보수성, 재사용성에서 뛰어나다.

첫 번째 설계는 단순한 시스템에서는 적합할 수 있으나, 확장성과 유지보수가 중요한 대규모 시스템에는 적합하지 않다.